

Confident Canis disposition
The model is converging cleanly on an answer.

Canis Apertus Lab

Inspectable local AI for Apertus

HackApertus Track 2B | Team: Canis Apertus Lab | Repository: github.com/UrsushoribilisMusic/canis-hackapertus-track2b

Vision. Canis brings the advantages of Apertus to people by making open weights visible in everyday use. The first benefit we expose is model disposition: because Apertus is open, Canis can inspect internal model signals and show whether an answer appears **confident, unsure, curious, concerned, reluctant, warm, or neutral**.

Product idea. Users should not only receive polished text; they should also receive an inspectable signal about how the model is answering. That signal is possible because the model is open, local, and available for inspection rather than hidden behind a closed API.

Prototype delivered. Canis includes a working iOS app using local MLX inference and a Dockerized browser demo for judges. The Docker package runs with `make run`; a real-model path uses `swiss-ai/Apertus-v1.5-8B`. The UI supports chat, local source upload, cited snippets, and "while Canis thinks" progress states for slow local Docker inference.

Judge test drive. Run `make run`, ask "What are truffles?", then upload a small PDF/text/Markdown file and ask a question about it. No API key or model download is required for this deterministic review path.

Judging criterion	What Canis demonstrates
Purposeful use of AI	Demonstrates the benefit of a Swiss open model by exposing information closed APIs do not provide: a local model-state signal alongside each answer.
Technical rigour	Forward-only J-space readout, controls against prompt-only artifacts, OOD tests, and inter-rater analysis.
Value, cost, scalability	Local inference reduces per-query cost, cloud traffic, and environmental footprint; the readout supports filtering, escalation, and review workflows.
Sovereign deployability	Goes beyond data residency: prompts, documents, model weights, and model inspection can run on-device or on-premise without transatlantic API traffic.
Implementation feasibility	Working iOS app, Docker demo, Apertus runtime path, source upload, tests, and reproducible commands.

1. User Experience

The user asks a question. Canis returns the answer plus a readout, for example:

Confident | 98% certain | "Truffles are a type of mushroom..."

The readout is not a mascot mood. It is a compact interface for model-state inspection, enabled by having open model weights available rather than relying on an opaque hosted API. In source mode, users add private PDFs, Markdown, or text files. Canis chunks them locally, retrieves relevant passages, and cites the snippets used for the answer. This turns a small local model into a private knowledge assistant while preserving an inspectability signal.

The iOS app is the product reference implementation. It uses local MLX inference on Apple silicon and is much faster than the portable Docker path. The Docker prototype prioritizes reproducibility for judges and on-premise deployment.

2. Architecture and Apertus Use

The submission has three layers:

1. Browser UI served from `src/web/`.
2. FastAPI backend exposing health, model, chat, source, and disposition endpoints.
3. Runtime layer selecting deterministic mock mode or the real Apertus path.

Default judge path:

```
make run
```

Real Apertus path:

```
export HF_TOKEN=hf_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
make run-model
```



Curious Canis disposition
The model is exploring context or opening a new line of thought.

The package does **not** bundle Apertus weights. The real path runs `swiss-ai/Apertus-v1.5-8B` through Swiss AI's Transformers branch, mounts the host Hugging Face cache, and attempts a hidden-state readout. If the model is already installed locally, Canis reuses it through the mounted cache. If not, and the user provides an authorized Hugging Face token, Transformers downloads the full gated model into that cache before the first answer. Once weights are present, runtime does not depend on a hosted inference API.

3. How the Disposition Readout Works

Canis taps the model residual stream at an internal layer and projects the hidden state into a vocabulary/logit-lens space: **J-space**. It then scores that vector against compact per-disposition directions and reports the strongest disposition plus a distribution over alternatives.

The six final product dispositions are:

Confident	converging cleanly on an answer	**Unsure**	plausible but less settled
Curious	exploring context or a new line	**Concerned**	detecting risk or sensitivity
Reluctant	holding back or avoiding a path	**Warm**	helpful, supportive, user-aligned

Neutral is shown when no disposition direction dominates.

The deployment property is important: inference is **forward-only**. There is no runtime backpropagation, no auxiliary neural network, and no sparse autoencoder training. The heavier step is one-time per-model calibration of small reference directions. On iOS, the readout engine is implemented around MLX tensors: it accepts a tapped hidden state and logits, projects the state with `jlens_projection.json`, cosine-scores it against `seed_vectors.json`, and computes normalized next-token entropy from logits. The current app keeps the same event channel even when the high-level MLX chat API does not expose every hidden-state tap, so the product path is ready for lower-level MLX access without changing the UI.

We did not trust a single attractive score. The evaluation checked:

- a prompt-only text classifier with no model access,
- out-of-distribution prompts with calibration frozen,
- inter-rater agreement for the labels,
- and geometry checks for direction stability and cross-axis entanglement.

One candidate axis, `mischief`, looked plausible in-distribution but failed the controls. It was removed from the product readout. That is a core part of the rigor story: Canis ships axes that survive validation, not every axis that looks interesting.

4. Evidence

In the companion disposition study, a discriminant J-space readout reached **89–95% macro recall** with **6–9% false-positive rate** under five-fold cross-validation across three small open models.

These experiments were calibrated on smartphone-class 3B–4B open models, including Apertus-4B, because that is the product constraint for the iOS app. The Docker runtime uses Apertus v1.5 8B to demonstrate the Track 2B deployment path; production deployments should keep a per-model calibration artifact for the exact deployed checkpoint.

Model	Naive seeds	Discriminant readout
Apertus-4B	33% recall / 21% FP	**89% recall / 7% FP**
Ministral-3B	27% recall / 18% FP	**95% recall / 9% FP**
Qwen3-4B	55% recall / 21% FP	**92% recall / 6% FP**

Out of distribution, frozen calibration achieved **79% / 80% / 70% recall** on Apertus / Ministral / Qwen. A prompt-only classifier reached **45%**; lexical baselines were **3–15%**. That gap is the central technical result: the readout is capturing more than keywords.

Four cross-family model judges agreed at **Fleiss' kappa = 0.89**; three humans agreed at **0.42** on the same 50-item protocol. The careful interpretation is that disposition is a model-native regularity, while some labels are more subjective for humans than for models.

5. Private Sources and Product Value



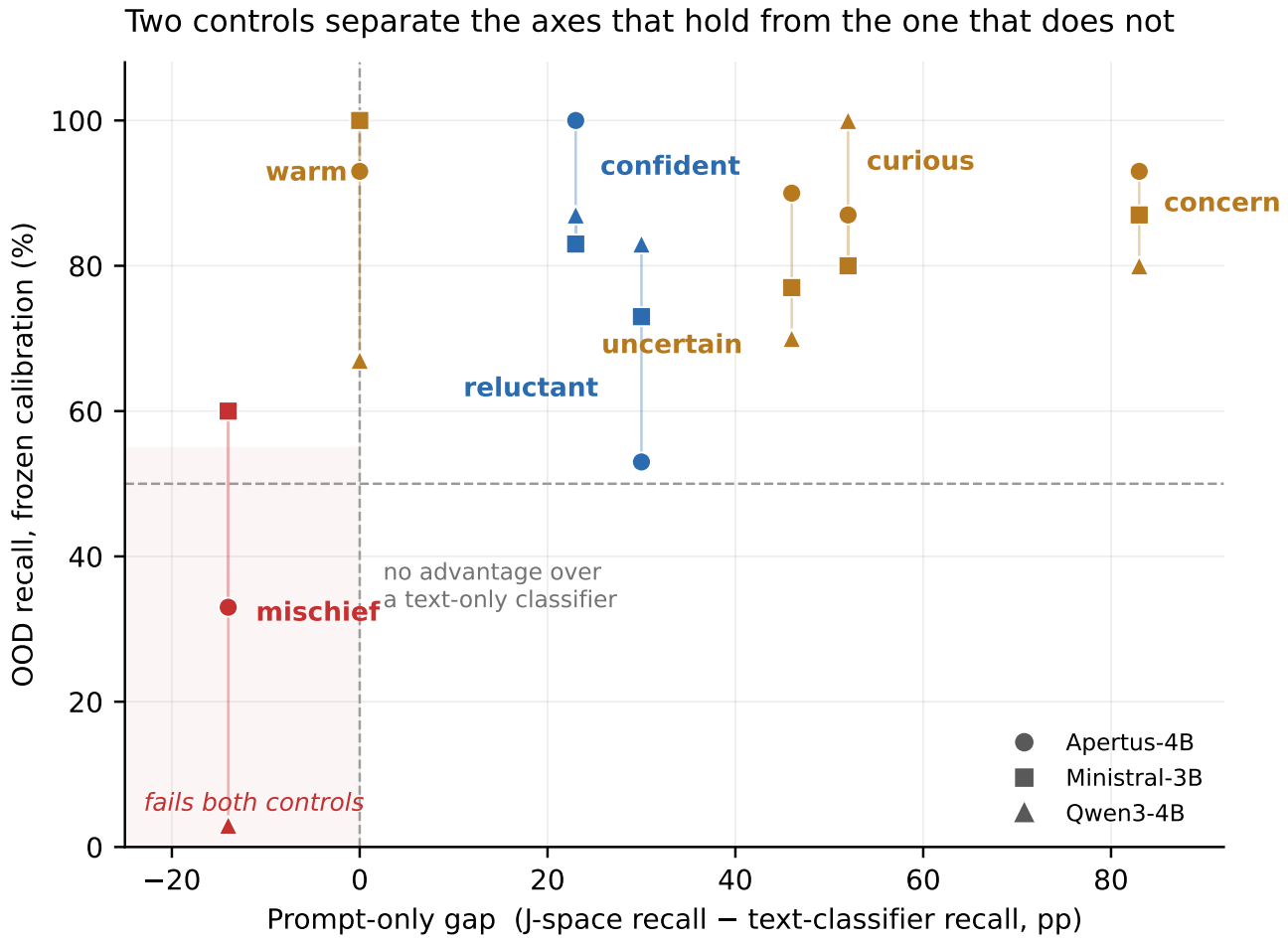


Figure 1: Controls separate model-state signal from prompt-only and lexical baselines.

The source flow demonstrates a practical use case: private document question answering. Users upload PDFs, Markdown, or text; the backend extracts text, chunks it locally, retrieves relevant passages, and passes only snippets into the answer context.

Commercially, the disposition signal can support:

- confidence-aware user interfaces that show when an answer should be treated cautiously,
- escalation of reluctant, concerned, or unsure answers to a human reviewer,
- review workflows for regulated or sensitive domains,
- filtering of low-certainty responses before publication,
- model-monitoring dashboards that aggregate disposition by topic, product surface, or model version,
- procurement and governance comparisons between local open models and hosted closed APIs,
- and public-facing education about how open models behave.

For enterprises, this is an additional parameter for understanding model behavior. It does not replace evaluations, source citations, or human review; it adds a runtime signal that can be logged with the answer and the supporting snippets. That makes Apertus more useful in corporate settings where explainability, audit trails, and operational controls matter. This strengthens the Apertus ecosystem because the feature depends on open-weight access. A closed API can return text; open weights let builders inspect hidden state and create new trust signals.

6. Cost, Scalability, and Sovereignty

Canis targets Track 2B architecture **a) on-premise**. The same design can be adapted to air-gapped or Swiss sovereign-cloud deployments because prompts, documents, and model weights can stay under the organization's control.

The iOS path avoids per-query cloud fees and avoids sending sensitive prompts to overseas servers. The Docker path gives judges and institutions a reproducible deployment target. The real-model Docker path is slower than iOS MLX on Apple silicon,



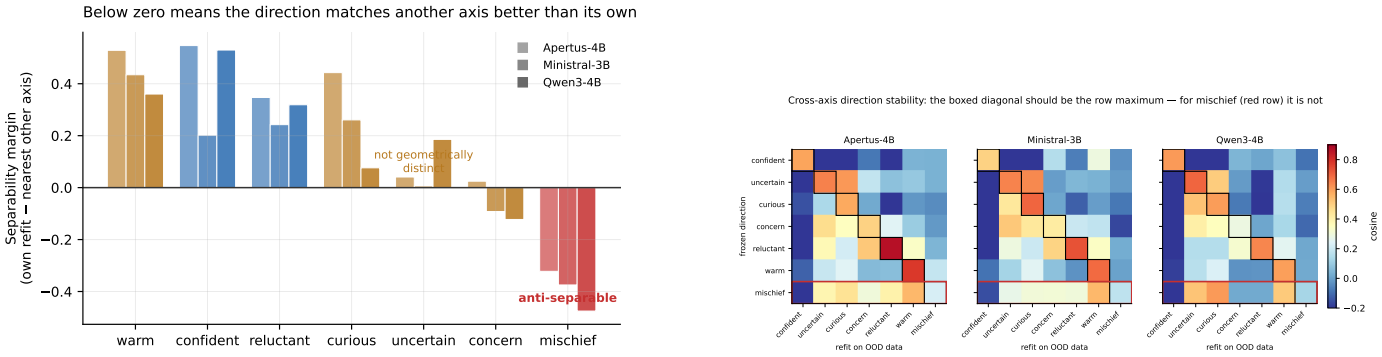


Figure 2: Disposition separability and cross-axis geometry from the companion analysis.

but it makes the system inspectable and portable.

7. Implementation Architecture and Team

Layer	Implementation
iOS product app	SwiftUI app using MLX Swift, MLXLMCommon, and MLXLLM. `ModelDownloadManager` downloads and stores MLX weights on-device; `CanisMLXEngine` manages model residency, streaming, memory, and thermal handling; `DispositionReadoutEngine` contains the forward-only hidden-state/logit readout. Local knowledge packs use PDFKit, Vision OCR fallback, chunking, and SQLite.
Docker backend	FastAPI/Python service in Docker. The default mode is deterministic for review; `make run-model` loads `swiss-ai/Apertus-v1.5-8B` with PyTorch/Transformers and attempts hidden-state scoring. Source upload uses local PDF/text/Markdown extraction and chunk retrieval.
Web interface	Browser UI served from `src/web/`. It mirrors the Canis interaction model: dog state, answer readout, source manager, history, settings, and "while Canis thinks" intermezzo content for slower Docker inference.

Team. Miguel Seglias led UI and interaction design. Raphael Anderegg and Luca Tescarini built core scaffolding and integration support. Miguel Angel Rodriguez Marquez led backend, disposition readout, source/RAG path, report, and submission coordination.

8. Enterprise Reuse Scenarios

The technology can be reused beyond the consumer app as an explainability layer for organizations running Apertus or related open models:

- **Corporate model explainability.** Add disposition telemetry to an internal Apertus deployment so teams can inspect not only what the model answered, but how settled or cautious the model appeared.
- **Governed knowledge assistants.** Combine source citations and disposition logs for internal manuals, compliance material, or technical support documents.
- **Support and escalation.** Route concerned, reluctant, or low-certainty answers to human review before they reach customers.
- **Model version comparison.** Track disposition distributions across prompts when evaluating a new local model, quantization, or fine-tune.
- **Sovereign private assistants.** Keep documents, prompts, model weights, and telemetry on-device or on-premise while still giving users an understandable trust signal.

9. Feasibility and Known Limits

Run and validate:



Warm Canis disposition
The response leans helpful, supportive, and user-aligned.

```
make run
make test
PORT=8081 make run
OPEN=0 make run
```

Real Apertus path:

```
export HF_TOKEN=hf_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
make run-model
```

10. What a Judge Can Verify Quickly

Try this	What it demonstrates
`make run`	Reproducible Docker startup, local browser UI, no hosted inference service for the default path.
Ask "What are truffles?"	A normal answer plus a disposition readout, e.g. confident with a high certainty score.
Upload a PDF/manual	Local chunking and retrieval; Canis answers from snippets instead of sending the whole document away.
Switch sources on/off	Difference between model-only answering and private-document grounding.
`make test`	Unit coverage for the resolver, source flow, and API contract.

Known limits. Docker inference on CPU/offload-heavy hardware can take from about a minute to several minutes per question. The source retrieval prototype uses lexical overlap rather than embeddings; an enterprise version should move to local BM25/embedding or hybrid retrieval while preserving the same on-premise boundary. The Docker readout falls back to lexical scoring only when the active local Transformers runtime cannot expose the expected hidden states/logits; this keeps `make run` reviewable, but the technical claim rests on the forward-only hidden-state/J-space method evaluated in the companion study. More domains and languages should be tested before high-stakes deployment.

References

Miguel A. Rodriguez Marquez. *Reading Disposition from Smartphone-Class Open Models: A Forward-Only J-Space Probe on Apertus-4B, Ministral-3B and Qwen3-4B*. 2026.
Swiss AI. `swiss-ai/Apertus-v1.5-8B`.
HackApertus Track 2B template and judging criteria.